PRÁCTICAS ALGORITMIA

Borja Cortés Alonso UO307315

ÍNDICE

Contenido

ÍNDICE 2
PRÁCTICA 0 3
PRÁCTICA 1.1 4
PRÁCTICA 1.2 10
PRÁCTICA 2 16
PRÁCTICA 3 20
PRÁCTICA 4 24
PRÁCTICA 5 25
PRÁCTICA 6 27
PRÁCTICA 7 28

PRÁCTICA 0

	Hasta 5000	Hasta 10000	Hasta 20000	Hasta 40000	Hasta 80000
PythonA1(Clase)	343	1353	5560	22328	89292
PythonA1(Casa)	914	3962	16019	64879	
JAVA SIN OPTIMIZACIÓN	169	592	2374	9764	37815
JAVA CON OPTIMIZACIÓN	58	146	580	2313	8902
JAVA EN ECLIPSE(OPTIMIZADO)	75	178	984	2609	9715
JAVA EN ECLIPSE SIN OPTIMIZAR	171	551	2195	9054	35505
PythonA2	117	444	1685	6406	24732
PythonA3	54	213	814	3283	11908
PythonA4	0	3	4	8	18
JavaA2	10	18	65	230	879
JavaA3	8	10	33	118	427
JavaA4	1	1	2	1	7

Tiempo en ms.

<u>Equipo utilizado en las mediciones</u>

Procesador: Intel(R) Core(TM) i5-6500T CPU @ 2.50GHz 2.50 GHz

RAM instalada: 16,0 GB

PRÁCTICA 1.1

TOMA DE TIEMPOS DE EJECUCIÓN

Se utiliza para ello el método currentTimeMillis () de la clase System del paquete de Java java. Ese método currentTimeMillis () devuelve un entero de tipo long (64 bits), que es el milisegundo actual que vive el ordenador (la cuenta a 0 se puso hace más 50 años: 1 de enero de 1970).

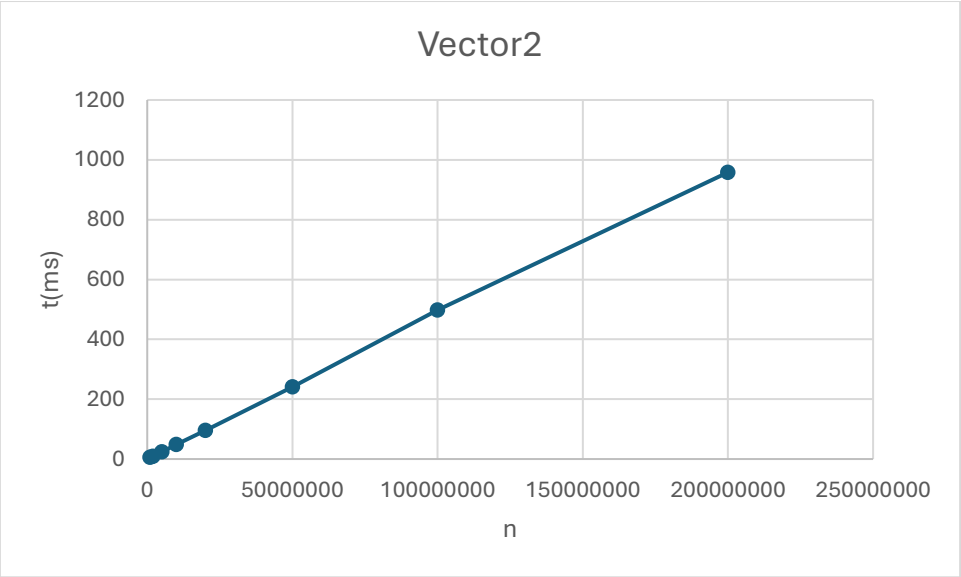
El valor máximo que puede representar un long es 2^63 - 1 ms. En años sería 2,9. 10^8 años.

Por lo tanto, se podrá seguir usando esta forma de contar durante **2,9. 10^8 años.**

5,8 x 10^8

Mediciones Vector2

n	Tiempo(ms)
1000000	6
2000000	9
5000000	24
10000000	48
20000000	96
50000000	241
100000000	498
200000000	958



¿Por qué a veces el tiempo medido sale 0?

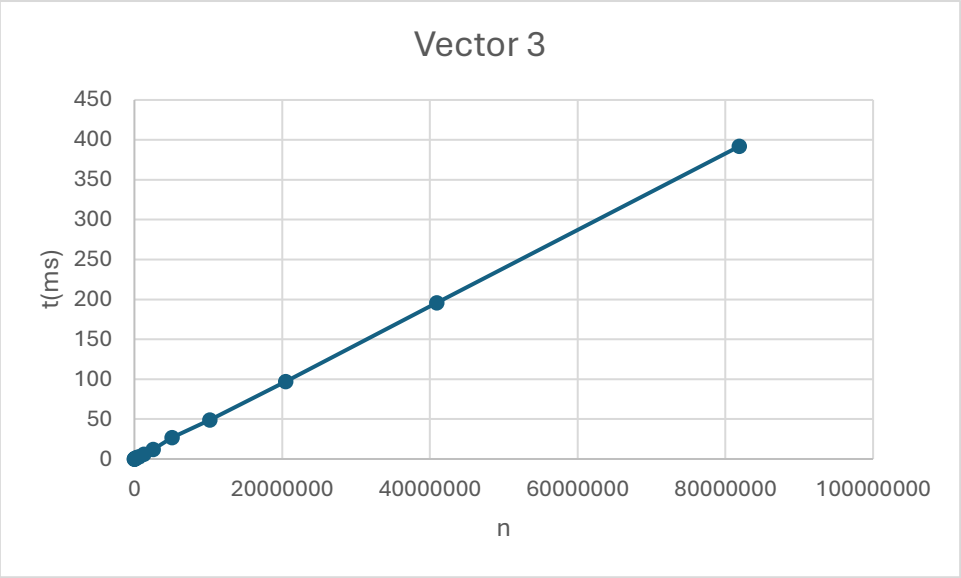
Porque el n no es suficientemente grande.

A partir de n = 1000000 se empiezan a obtener tiempos fiables.

Mediciones Vector3

```
n   tTiempo
10000   0
20000   0
40000   0
80000   1
160000  1
320000  2
640000  3
1280000 6
2560000 12
5120000 27
10240000        49
20480000        97
40960000        196
81920000        392
```

n	Tiempo(ms)
10000	0
20000	0
40000	0
80000	1
160000	1
320000	2
640000	3
1280000	6
2560000	12
5120000	27
10240000	49
20480000	97
40960000	196
81920000	392



En el Vector3 las mediciones por debajo de 50 ms no son fiables.

Mediciones Vector 4

n	Tiempo(ms)
10000	0.11663
20000	0.22585
40000	0.45124
80000	0.88732
160000	1.7734
320000	3.43
640000	7.18
1280000	14.06
2560000	28.48
5120000	57.74
10240000	113.44
20480000	227.23
40960000	479.44
81920000	926.14

<u>¿Qué pasa con el tiempo si el tamaño del problema se multiplica por 2?</u>

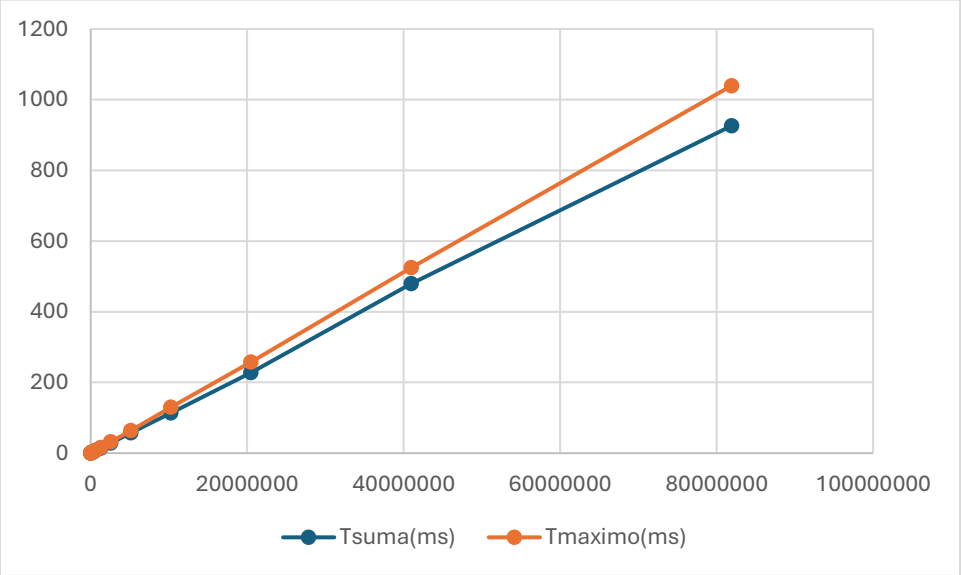
Al duplicar el tamaño el tiempo de ejecución también se duplica (hay un cierto error asociado). Esto se debe a la complejidad lineal del algoritmo.

<u>¿Qué pasa con el tiempo si el tamaño del problema se multiplica por otro k que no sea 2?</u> (Pruebe, por ejemplo, para k=3 y k=4 y compruebe los tiempos obtenidos.) Razone si los tiempos obtenidos son los que se esperaban de la complejidad lineal O(n).

Si el tamaño del problema se multiplica por otro k, el tiempo obtenido también se multiplicaría por k. Los tiempos obtenidos son los esperados según la complejidad O(n) ya que los tiempos crecen de forma proporcional al tamaño.

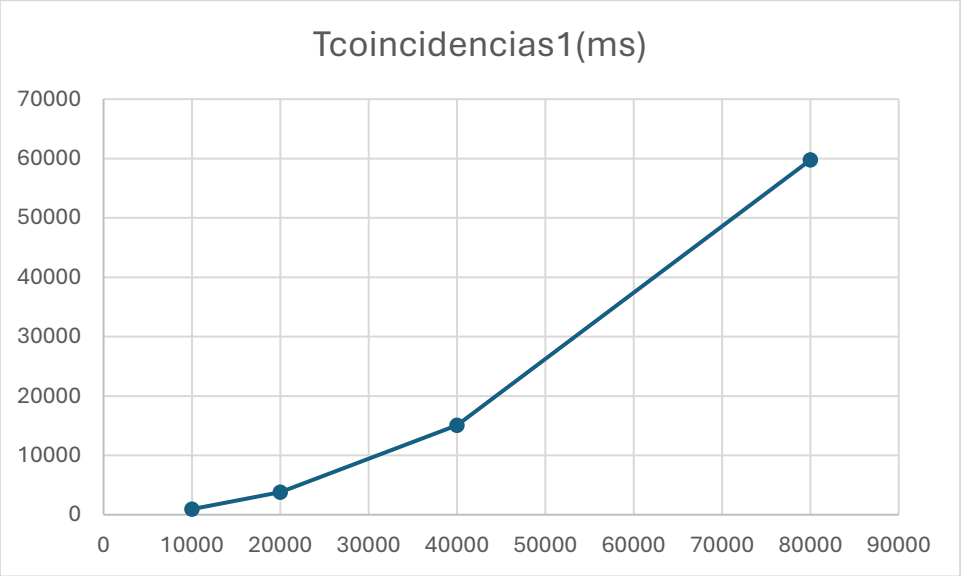
Medición de tiempos Vector5, Vector6 y Vector7:

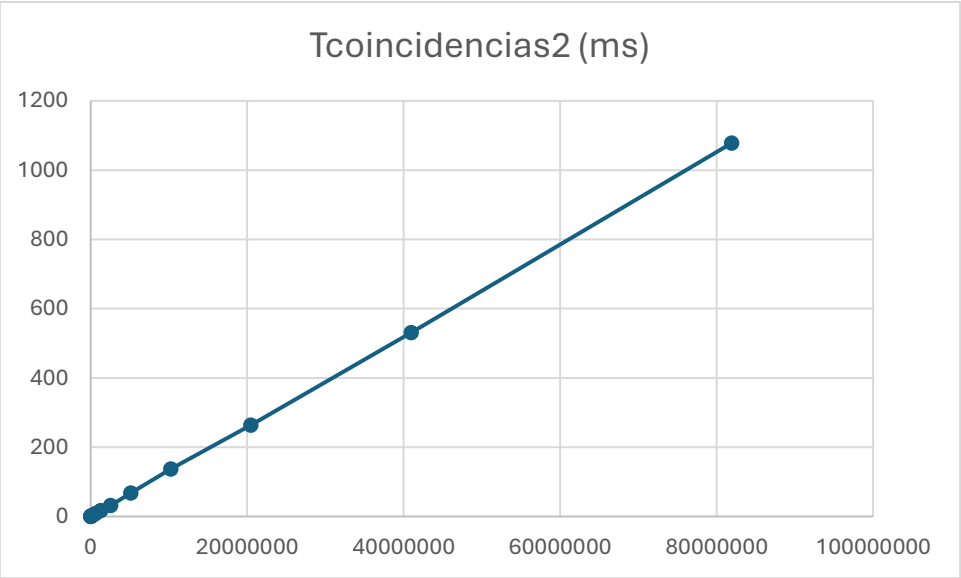
n	Tsuma(ms)	Tmaximo(ms)
10000	0,11663	0,12704
20000	0,22585	0,25054
40000	0,45124	0,49714
80000	0,88732	1,01856
160000	1,7734	2,02031
320000	3,43	4,10931
640000	7,18	7,97
1280000	14,06	16,1
2560000	28,48	32,2
5120000	57,74	64,0
10240000	113,44	129,7
20480000	227,23	257,1
40960000	479,44	525,0
81920000	926,14	1039,4



n	Tcoincidencias1(ms)	Tcoincidencias2 (ms)
10000	938	0,1355
20000	3796	0,2674
40000	15047	0,5204
80000	59750	1,03
160000	FdT	2,18
320000	FdT	4,07

640000	FdT	8,43
1280000	FdT	17,08
2560000	FdT	32,2
5120000	FdT	67,96
10240000	FdT	137,53
20480000	FdT	264
40960000	FdT	531
81920000	FdT	1078





Los tiempos de ejecución de Vector5(suma) y Vector6(máximo) crecen de proporcional al tamaño del problema n. Este hecho confirma que ambos algoritmos tienen una complejidad O(n).

El algoritmo coincidencias1 presenta complejidad cuadrática tal y como se puede apreciar en la gráfica. Por lo tanto, la diferencia respecto a coincidencias2(complejidad lineal) se va haciendo cada vez más notable.

Equipo utilizado en las mediciones

Procesador: Intel(R) Core(TM) i5-6500T CPU @ 2.50GHz 2.50 GHz

RAM instalada: 16,0 GB

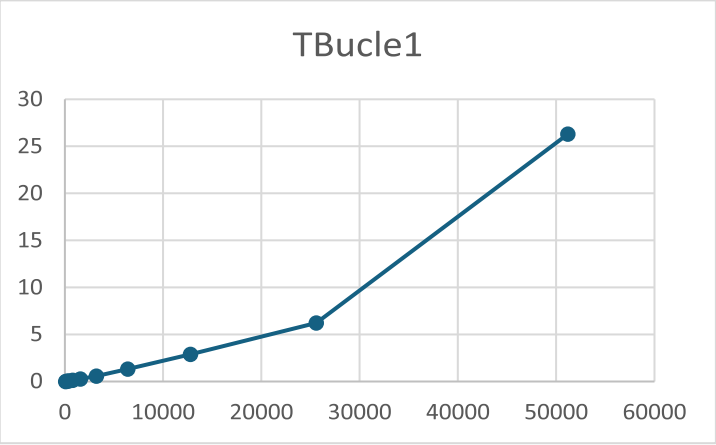
PRÁCTICA 1.2

1)ALGUNOS MODELOS ITERATIVOS

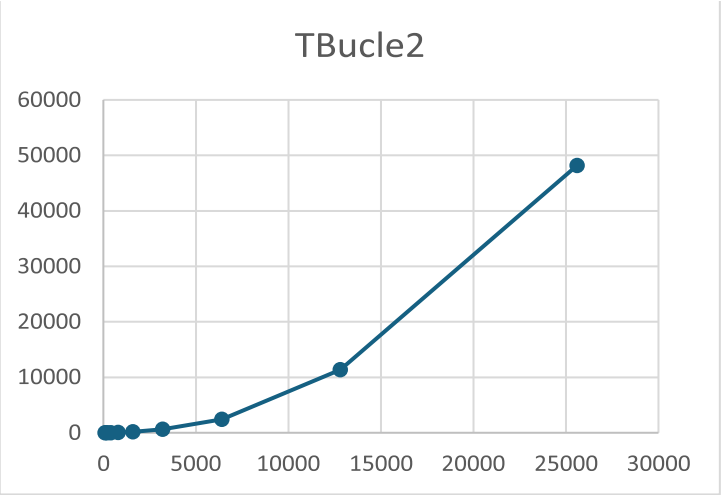
n	TBucle1	TBucle2	TBucle3	TBucle4
100	0,0137	0,55	1,52	1,4
200	0,0333	1,57	6,49	13,8
400	0,0697	7,69	28,23	85,2
800	0,127	32,6	127	659,5
1600	0,285	158	519	10673
3200	0,584	630	2180	FDT
6400	1,33	2444	9875	FDT
12800	2,889	11356	75854	FDT
25600	6,211	48174	FDT	FDT
51200	26,311	FDT	FDT	FDT

Tiempos medidos en ms.

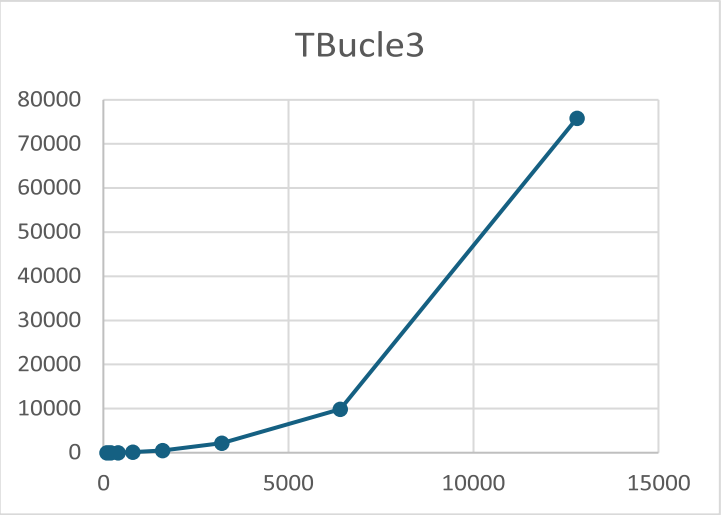
Bucle1: O(n log(n))



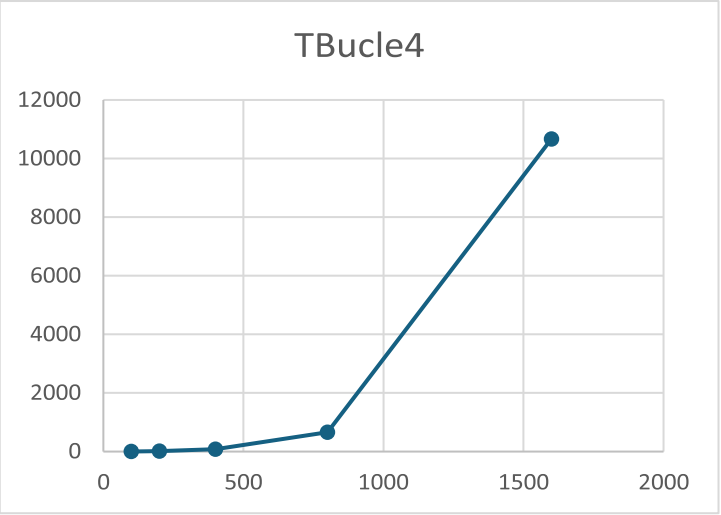
Bucle2: O(n^2 log n)



Bucle3: $O(n^2 \log n)$



Bucle4: $O(n^3)$



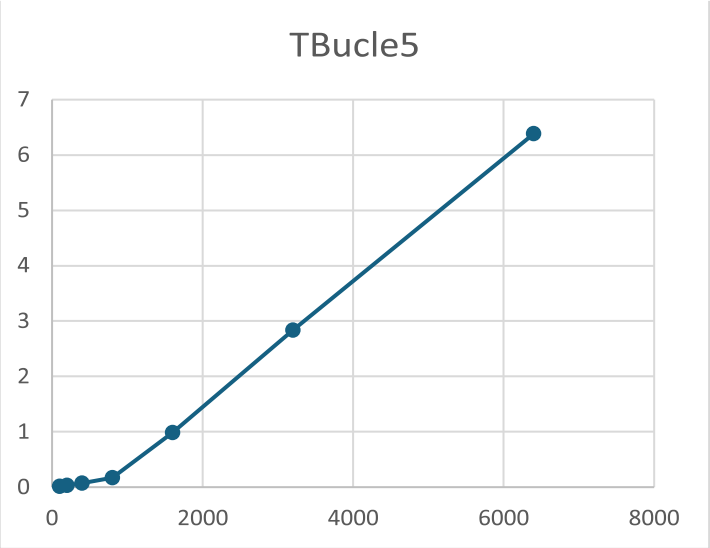
Los tiempos obtenidos son coherentes teniendo en cuenta la complejidad de cada algoritmo.

2)CREACIÓN DE MODELOS ITERATIVOS DE UNA COMPLEJIDAD DADA

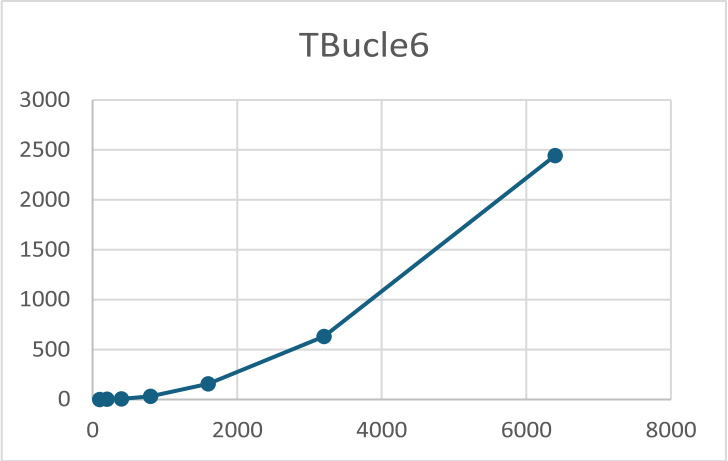
n	TBucle5	TBucle6	TBucle7
100	0,0137	0,55	1,52
200	0,0333	1,57	6,49
400	0,0697	7,69	28,23
800	0,1678	32,6	127
1600	0,9873	158	519
3200	2,8345	630	2180
6400	6,3837	2444	9875

Tiempos medidos en ms.

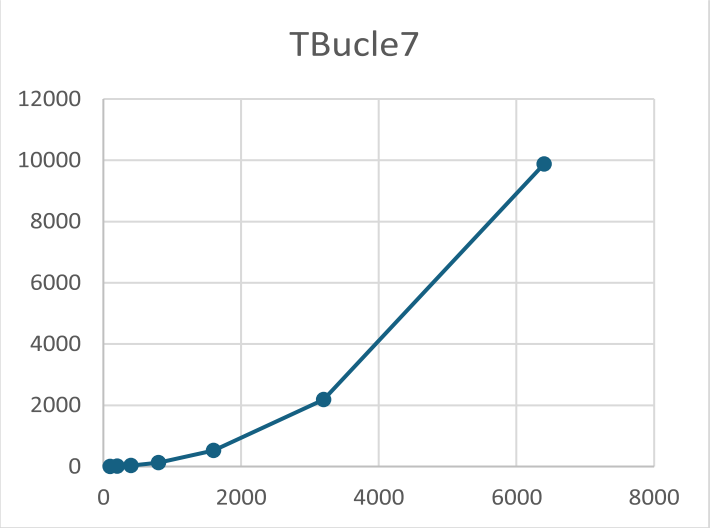
Bucle5: O(n2log 2 n)



Bucle6: O (n3 log n)



Bucle7: O (n4)



Los tiempos obtenidos se corresponden con la complejidad teórica de los algoritmos.

3)RELACION TEMPORAL DE ALGORITMOS

CASO1: DOS ALGORITMOS DE DIFERENTE COMPLEJIDAD

n	TBucle1(t1)	TBucle2(t2)	t1/t2
100	0,0137	0,55	0,02490909
200	0,0333	1,57	0,02121019
400	0,0697	7,69	0,00906372
800	0,127	32,6	0,00389571
1600	0,285	158	0,0018038
3200	0,584	630	0,00092698

6400	1,33	2444	0,00054419
12800	2,889	11356	0,0002544
25600	6,211	48174	0,00012893
51200	26,311	FDT	

Podemos observar que el cociente entre t1 y t2 tiende a 0, el resultado es el esperado ya que se divide un algoritmo de una complejidad menor entre uno de una complejidad mayor.

CASO2: DOS ALGORITMOS CON LA MISMA COMPLEJIDAD

n	TBucle3(t3)	TBucle2(t2)	t3/t2
100	1,52	0,55	2,76363636
200	6,49	1,57	4,13375796
400	28,23	7,69	3,6710013
800	127	32,6	3,89570552
1600	519	158	3,28481013
3200	2180	630	3,46031746
6400	9875	2444	4,04050736
12800	41880	11356	3,68791828
25600	FDT	48174	
51200	FDT	FDT	

Al ser algoritmos con la misma complejidad, los tiempos evolucionan de la misma forma, aunque difieren en una constante. En este caso, el bucle2 proporciona mejores resultados.

CASO3: MISMO ALGORITMO EN ENTORNOS DE DESARROLLO y/o EJECUCIÓN DIFERENTES

n	Bucle4Python	Bucle4-Java SIN OPTIMIZACION	tPython/tJava
100	12	1,4	8,57142857
200	99	13,8	7,17391304
400	548	85,2	6,43192488
800	4270	659,5	6,47460197
1600	46907	10673	4,39492177

3200	FDT	FDT	
6400	FDT	FDT	
12800	FDT	FDT	
25600	FDT	FDT	
51200	FDT	FDT	

Podemos observar que los tiempos evolucionan de la misma forma aunque con la diferencia de una constante (aproximadamente entre 6 y 7).En este caso sería recomendable realizar la ejecución en Python.

Equipo utilizado para realizar las mediciones:

Procesador: Intel(R) Core(TM) i5-8250U CPU @ 1.60GHz (1.80 GHz)

Memoria RAM: 16,0 GB

PRÁCTICA 2

TABLA 1= ALGORITMO BURBUJA

n	t ordenado (ms)	t inverso (ms)	t aleatorio (ms)
10000	760	1625	1344
20000	2937	6640	5484
40000	11872	26235	21703
80000	49664	FdT	FdT
160000	FdT	FdT	FdT
320000	FdT	FdT	FdT
640000	FdT	FdT	FdT
1280000	FdT	FdT	FdT

Los tiempos obtenidos concuerdan con la complejidad del algoritmo O(n2) que se mantiene para todos los casos, las diferencias residen en el número de intercambios, no en la complejidad.

TABLA 2= ALGORITMO SELECCIÓN

n	t ordenado (ms)	t inverso (ms)	t aleatorio (ms)
10000	656	704	672
20000	2672	2813	2755
40000	11068	11516	10766
80000	43281	45563	42327
160000	FdT	FdT	FdT
320000	FdT	FdT	FdT
640000	FdT	FdT	FdT
1280000	FdT	FdT	FdT

La complejidad teórica vuelve a ser O(n2) al igual que en el algoritmo burbuja. Además, el número de comparaciones es el mismo en todos los casos por lo que las diferencias no son notables.

TABLA 3= ALGORITMO INSERCIÓN

n	t ordenado (ms)	t inverso (ms)	t aleatorio (ms)
10000	0,33	1063	531
20000	0,578	4388	2172
40000	1,205	17247	8704
80000	2,471	FdT	34239
160000	4,816	FdT	FdT
320000	10,399	FdT	FdT
640000	19,682	FdT	FdT
1280000	38,405	FdT	FdT

Los tiempos concuerdan con la complejidad teórica, para el caso del vector ordenado la complejidad es O(n) y para el resto de casos la complejidad es de O(n2).

TABLA 4= ALGORITMO RÁPIDO

n	t ordenado (ms)	t inverso (ms)	t aleatorio (ms)
10000	4,38	4,53	5,62
20000	9,06	9,85	14,08
40000	19,53	21,86	28,12
80000	42,15	46,4	60,84
160000	86,56	96,23	127,03
320000	182,07	199,57	263,75
640000	379,97	416,72	551,34
1280000	789,09	850,25	1162,26

El algoritmo rápido presenta la misma complejidad O(n log n) para los 3 casos por lo tanto los datos de corresponden con la complejidad teórica. También es cierto que el peor caso O(n2) no aparece porque el pivote es el elemento central.

¿Cuántos días (1 día= 86 400 000 milisegundos) tardaría cada uno de los cuatro métodos (Burbuja, Selección, Inserción y Rápido) en ordenar un vector de n= 81 920 000 (o sea, n=80000*24*26=80000*210), en el caso inicialmente en orden aleatorio?

A partir de los datos obtenidos en las tablas anteriores tenemos:

Burbuja

Sabiendo que para n=40 000 tenemos 21 707 ms, aproximadamente para n=80000 tenemos 86 812 ms debido a la evolución cuadrática.

Por lo tanto, los datos que tenemos son:

n1= 80 000 n2 = 81 920 000 y t1 = 86 812 ms. Para calcular t2:

$$t2 = t1 * \left(\frac{n2}{n1}\right)^2$$

t2 =91 000 000 000 ms = **1053 días**

Selección

Los datos que tenemos son:

n1= 80 000 n2 = 81 920 000 y t1 = 42327 ms. Para calcular t2:

$$t2 = t1 * \left(\frac{n2}{n1}\right)^2$$

t2 =44 394 000 000 ms = **513,7 días**

Inserción

Los datos que tenemos son:

n1= 80 000 n2 = 81 920 000 y t1 = 34239 ms. Para calcular t2:

$$t2 = t1 * \left(\frac{n2}{n1}\right)^2$$

t2= 35 865 000 000 ms = **415 días**

Rápido

Los datos que tenemos son:

n1= 80 000 n2 = 81 920 000 y t1 = 60,84 ms. Para calcular t2:

$$t2 = t1 * (n2 * \log(n2)/n1 * \log(n1))$$

t2= 100 500 ms = **0,00116 días**

TABLA 5= RÁPIDO vs. INSERCIÓN PARA TIEMPOS BAJOS

Tiempos medidos para 100000 repeticiones.

n	t rápido (ms)	t inserción (ms)	t rápido / t inserción
10	184	101	1,82178218
15	400	157	2,5477707
20	441	291	1,51546392
25	668	546	1,22344322
30	811	665	1,21954887
35	985	831	1,18531889
40	1121	1138	0,98506151
45	1298	1188	1,09259259
50	1521	1420	1,07112676
55	1718	1841	0,93318848
60	1832	2328	0,78694158

65	2229	2651	0,84081479
70	2149	2827	0,76016979
75	2388	3398	0,70276633
80	2791	3926	0,71090168
85	2820	4190	0,67303103
90	2854	4800	0,59458333
95	3181	5357	0,5938025
100	3337	5890	0,56655348

Podemos observar que a partir de aproximadamente n = 50 hay un cambio de tendencia y los tiempos usando el algoritmo rápido son sustancialmente más bajos según aumenta n.

Creando una nueva clase (RapidoInsercionTiemposBajos) que combina el algoritmo de inserción y el algoritmo rápido obtenemos el tiempo más bajo posible independientemente de la n, a partir de n = 50 empieza a tener los mismos tiempos que con el algoritmo Rápido.

<u>Equipo utilizado para las mediciones:</u>

Procesador: Intel(R) Core(TM) i5-6500T CPU @ 2.50GHz 2.50 GHz

RAM : 16,0 GB

PRÁCTICA 3

Tras analizar la complejidad de las tres clases anteriores, no se le pide hacer sus tablas de tiempos, pero sí razonar si los tiempos coinciden o no la complejidad temporal de cada algoritmo.

En Sustracción1 la complejidad teórica es O(n), sus tiempos se duplican cuando se duplica n por lo tanto se los tiempos se corresponden con la complejidad. En Sustracción2 también se corresponden los tiempos con su complejidad (O(n^2)).Por último, en Sustracción3 tenemos una complejidad O(2^n) y unos tiempos que se duplican cuando n aumenta en una unidad, por lo tanto, son coherentes.

Contestar: ¿para qué valor de n dejan de dar tiempo (abortan) las clases Sustracción1 y Sustracción2? ¿por qué sucede eso?

Sustracción1 aborta para n=16000 y Sustracción2 para n=16000, se produce un StackOverflowError, es decir, un desbordamiento en la pila que ocurre por la cantidad de llamadas recursivas.

Calcular: ¿cuántos años tardaría en finalizar la ejecución Sustracción3 para n=80? Razonar la respuesta.

Teniendo en cuenta la complejidad O(2^n) de Sustracción3 y sabiendo que para n1 =20 t1 =3428 ms para n2 =80

31 millones de años.

Implementar una clase Sustracción4.java con una complejidad O(n3) (con su pertinente toma de tiempos) y después rellenar una tabla en la que se muestre el tiempo (en milisg.) para n=100, 200, 400, 800, … (hasta FdT).

n	Sustracción4(ms)
100	6
200	51
400	413
800	3457
1600	26344
3200	FdT
6400	FdT

Implementar una clase Sustracción5.java con una complejidad O(3n/2) (con su pertinente toma de tiempos) y después rellenar una tabla en la que se muestre el tiempo (en milisg.) para n=30, 32, 34, 36, … (hasta FdT).

n	Sustracción5(ms)
30	804
32	2404
34	7599
36	22530
38	FdT
40	FdT

Tras analizar la complejidad de las tres clases anteriores, no se le pide hacer sus tablas de tiempos, pero sí razonar si los tiempos coinciden o no la complejidad temporal de cada algoritmo.

El algoritmo Division1 presenta complejidad lineal.

El algoritmo División2 presenta complejidad O(n log(n)).

El algoritmo Division3 presenta complejidad O(n).

Los tres algoritmos tienen tiempos acordes a su complejidad.

Tiempos Division4 (en ms)

n	Division4
1000	31
2000	83
4000	344
8000	1375
16000	5501
32000	22046
64000	FdT

Tiempos Division5 (en ms)

n	Division5
1000	63
2000	246

4000	1000
8000	4243
16000	16479
32000	FdT
64000	FdT

Fibonacci

Los métodos 1 y 2 son iterativos con una complejidad O(n), el método 3 es un método recursivo con sustracción (O(n)) y el método4 es un método recursivo con una complejidad exponencial es por eso que los tiempos son mucho más elevados.

Método1	Método2	Método3	Método4
0,00062	0,000796	0,001046	8141

Eficiencia: metodo1>metodo2> método3 >>>> método4

VectorSuma

El método1 es iterativo, el método 2 es recursivo con sustracción y el método 3 es recursivo con división.

Método1	Método2	Método3
0,001141	0,00261	0,00575

Eficiencia: metodo1>metodo2> método3

<u>PRÁCTICA DyV</u>

PuntosTrivialTiempo

n	Tiempo(en ms)
1024	16
2048	16
4096	31
8192	125
16384	485
32768	1937
65536	7781
131072	34913
262144	FdT

Los tiempos concuerdan con la complejidad del algoritmo O(n^2).

PuntosFyDTiempos

n	PuntosFyD
1024	31
2048	0
4096	15
8192	16
16384	31
32768	62
65536	141
131072	156
262144	297
524288	672
1048576	1532
2097152	3584
4194304	8312
8388608	16840
16777216	39796
33554432	FdT

El algoritmo DyV tiene una complejidad de O(nlog(n)), los tiempos concuerdan perfectamente.

PRÁCTICA 4

TABLA TIEMPOS ALGORITMO DEVORADOR

n	Tiempo(ms)
8	0,0112
16	0,0129
32	0,015
64	0,0161
128	0,0401
256	0,1
512	0,23
1024	0,49
2048	1,03
4096	2,71
8192	5,76
16384	13,91
32768	30,31
65536	78,37

Los tiempos obtenidos confirman una complejidad lineal O(n) que concuerda con el código implementado. Al procesar cada nodo una sola vez y asignarle el primer color disponible, el tiempo de ejecución escala proporcionalmente al número de regiones.

PRÁCTICA 5

Definición de estado

El estado de la tabla se guarda en una matriz de valores booleanos denominada dp[i][l]. i representa el número de vehículos en la cola y l la longitud ocupada en babor.

Cada valor booleano de la tabla muestra si es posible cargar los primeros i vehículos de modo que se ocupen l posiciones de babor.

Relación de concurrencia

Caso base:

- dp[0][0] = true. Si no hay vehículos no se ocupa nada

Transiciones:

- A babor, si el espacio actual en babor (l) más el nuevo vehículo no supera el límite:
- A estribor, si el espacio ocupado allí (calculado como la suma total hasta ahora menos lo que hay en babor) no supera el límite.

Complejidad

La complejidad es O(N*L) ya que tenemos un bucle anidado que recorre cada vehículo (N) y, para cada uno, recorre las posibles longitudes del carril (L).

Implementación

```
public void run(String file){
        loadData(file);
        int n = vehicles.size();
    dp = new boolean[n + 1][boatLength + 1];
    sumatorio = new int[n + 1];

    sumatorio[0] = 0;
```

```
for (int i = 1; i <= n; i++) {
sumatorio[i] = sumatorio[i - 1] + vehicles.get(i - 1);
}
        this.dp[0][0] = true;
        for(int i=1;i <= vehicles.size();i++){
                for(int l=boatLength; l >=0; l--){
                        if(!dp[i-1][l]){
                                continue;
                        }

                        //meter coche en babor
                        if((l+vehicles.get(i-1))<=boatLength){
                                dp[i][l +vehicles.get(i-1)] = true;
                        }

                        //meter coche en estribor
                        if((sumatorio[i]-l)<=boatLength){
                                dp[i][l]=true;
                        }
                }
        }
}
```

PRÁCTICA 6

Tiempos AlmacenajeContenedores

Fichero	Tiempo(ms)
Test00	0
Test01	0
Test02	0
Test03	0
Test04	0
Test05	0
Test06	1
Test07	1
Test08	139
Test09	FdT

La complejidad del algoritmo es exponencial,O(n!) en el peor caso.

Esto se debe a que utiliza backtracking, probando todas las posibles formas de asignar los n objetos a contenedores: en cada paso puede colocar el objeto en cualquiera de los contenedores existentes o crear uno nuevo, generando un número de combinaciones del orden de n!.

Aunque incluye poda para reducir casos en la práctica, no evita el crecimiento exponencial en el peor caso.

PRÁCTICA 6 BIS

En esta práctica se ha implementado una solución al problema de las N-Reinas utilizando backtracking. El objetivo es colocar N reinas en un tablero de ajedrez de NxN sin que ninguna se ataque entre sí; es decir, que no compartan fila, columna ni diagonal.

Para ello, se utiliza una función recursiva que intenta posicionar una reina por fila, avanzando hacia la siguiente solo si la casilla actual es segura.

Si la columna o diagonal está ocupada, el algoritmo "corta" esa rama y no pierde tiempo entrando en ella.

PRÁCTICA 7

Heurístico de ramificación

Antes de empezar se ordenan los objetos que se van a meter en los contenedores de mayor a menor para que la intentar meter primero los que más espacio ocupan se mejora el tiempo.

En el método backtracking se hace una poda por cota inferior en la que se calcula lo que ocupan los objetos restantes y los contenedores mínimos necesarios. Si los contenedores necesarios más los ya usados son mayor o igual al mejor resultado hasta el momento se 'poda' esa rama.

Comparación de tiempos

Hay una clara diferencia en los tiempos sobre todo a partir del test08 y el test09 que en la p6 tardaban 139ms y FdT.

Los demás tiempos son 0 ms en las dos implementaciones ya que en la implementación de la práctica 6 también se hizo ordenación al principio y una poda en el backtracking (más básica